

Email-Based OTP (Replacing SMS/DLT for v1 Launch)

Scope: this addendum covers only what changed. All other sections of the BRD and Developer Specification remain in effect.

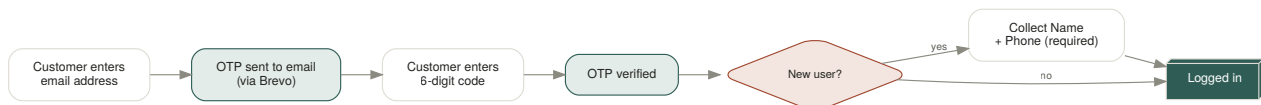
Why this change

SMS-based OTP in India requires DLT (Distributed Ledger Technology) registration — a multi-step regulatory process (Principal Entity registration, Sender ID/Header approval, Content Template approval) taking roughly 3-7 days and a one-time fee, before a single OTP SMS can be sent. To avoid this becoming a launch blocker, authentication and all customer notifications move to **email** for v1. Email requires no equivalent registration in India.

Kept swappable: all sending goes through one NotificationChannel interface. Moving back to SMS later — once DLT is complete — is a config flag change, not a rewrite.

Updated Auth Flow

Login identifier changes from phone number to email address. Phone number is still collected, but only as a required contact field for delivery coordination — it is never verified and never used to log in.



What changed, at a glance

Login identifier

Phone + OTP → Email + OTP. Email is now unique and required on the users table.

Phone number

No longer the identifier. Still collected (required) at first login, but purely as delivery contact info — unverified.

OTP delivery

MSG91 (SMS) → Brevo (email). No DLT registration needed.

Order notifications

All notifications (OTP, order confirmation, status updates) now route through email via the same provider, instead of SMS.

Redis keys

otp:{phone} → otp:{email}; same for the rate-limit key.

Environment variables

MSG91 variables removed; Brevo variables added (see below).

Updated users Table

Field	Type	Change
id	uuid	unchanged — PK
email	string	now unique, required — the login identifier (was optional, receipts-only)
email_verified_at	timestamp	new — replaces phone_verified_at; set on first successful OTP verification
phone	string	still required , but now contact-only — not unique, not verified, not used for login
name	string	unchanged — required, collected at first login

Notification Channel Abstraction

One interface, swapped by a single environment variable. Nothing else in the auth or order-notification logic needs to know which provider is behind it.

```
interface NotificationChannel {
  send(destination: string, message: string): Promise<void>;
}

class EmailChannel implements NotificationChannel {
  // Active now — uses Brevo's transactional email API
  async send(destination: string, message: string) {
    await brevoClient.sendTransacEmail({
      sender: { email: process.env.EMAIL_FROM_ADDRESS },
      to: [{ email: destination }],
      subject: 'Your verification code',
      htmlContent: `<p>${message}</p>`
    });
  }
}

class SmsChannel implements NotificationChannel {
  // Added later, once DLT registration is complete — not built in v1
  async send(destination: string, message: string) {
    await msg91Client.sendSms(destination, message);
  }
}

// .env
NOTIFICATION_CHANNEL=email // flip to "sms" later — zero code changes elsewhere

const notifier: NotificationChannel =
  process.env.NOTIFICATION_CHANNEL === 'sms' ? new SmsChannel() : new EmailChannel();
```

Redis Key Structures (Updated)

Key pattern	Value	TTL
otp:{email}	{ otp_hash, attempt_count }	5 min
ratelimit:otp:{email}	send count	10 min
refresh:{token_hash}	user_id	30 days

Environment Variables (Updated)

```
# Email (Brevo) – replaces MSG91 for v1
EMAIL_PROVIDER=brevo
BREVO_API_KEY=          # keep in Render's env var dashboard – never commit this
EMAIL_FROM_ADDRESS=     # your verified Brevo sender email or domain address
NOTIFICATION_CHANNEL=email

# MSG91 variables removed for v1 – re-add only once DLT registration is complete
# MSG91_AUTH_KEY=
# MSG91_SENDER_ID=
# MSG91_OTP_TEMPLATE_ID=
```

Security note: if a Brevo API key is ever pasted into chat, committed to git, or shared in a screenshot, treat it as compromised — revoke it in Brevo (SMTP & API → API Keys) and generate a fresh one immediately. Store only in Render's environment variable dashboard.

Impact on Phase 2 (SMS re-introduction)

When DLT registration eventually completes, re-enabling SMS is limited to: implementing `SmsChannel` against MSG91 (already scaffolded above), restoring the MSG91 environment variables, and flipping `NOTIFICATION_CHANNEL=sms`. No changes are needed to OTP generation, hashing, Redis storage, rate limiting, or any order-notification trigger points — all of that logic is already channel-agnostic.